ZEDPATH

Team Kestrel · IntelliCon '26 Buildathon

ZP-DOC-06

Software Architecture

Context, containers, components, runtime flows, security, free-plan budget and architecture decisions

Version	0.9
Status	In review
Date	11 October 2026
Classification	Public
Document owner	B.M.P Banneka, Product Owner

© 2026 Team Kestrel. Published under the classification shown above.

Uncontrolled when printed or exported. The controlled version is the Markdown source in the repository: docs/06-architecture/ZP-DOC-06_Architecture.md

Document control

Document information

Document ID	ZP-DOC-06
Title	Software Architecture
Version	0.9
Status	In review
Classification	Public
Document owner	B.M.P Banneka, Product Owner
Author	Team Kestrel (drafted with Claude, AI coding partner)
Conforms to	C4 model (levels 1 to 3) for structure; UML 2.5.1 sequence diagrams for behaviour; Architecture Decision Records (Nygard format); ZP-DOC-00
Source file	docs/06-architecture/ZP-DOC-06_Architecture.md
Repository	https://github.com/PraveenBanneka/zedpath

Revision history

Version	Date	Author	Description of change
0.9	11 Oct 2026	Team Kestrel	First issue: architecture as deployed, verified against wrangler.jsonc, worker/, src/, shared/ and migrations 0001 to 0010; six ADRs; free-plan budget; submitted for approval

Review and approval

This document takes effect when the approver signs below. Until then its status is as shown on the cover.

Role	Name	Signature	Date
Reviewer	Claude (AI coding partner), consistency and traceability review		
Approver	B.M.P Banneka, Product Owner		

Contents

1 Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 Intended audience 5
1.4 Relationship to other documents 5
2 Architectural drivers 6
2.1 Constraints 6
2.2 Quality attributes 6
3 System context 8
4 Container and deployment view 9
5 Component view 11
5.1 Worker modules 11
5.2 Client screens 11
5.3 Shared rule engine 11
6 Key runtime flows 12
6.1 Results to bands 12
6.2 Ask with citations 12
6.3 Joining a group, rules accepted once 13
6.4 Journey step window (409) 13
7 Security 14
8 Free-plan budget 15
9 Architecture decision records 16
10 Deployment and rollback 18
11 Risks 19
Appendix A: Traceability: requirements to architecture 20
Appendix B: Glossary 21
Appendix C: References 22

List of figures

Figure 1: C4 container view of ZedPath as deployed 9
Figure 2: Ask ZedPath: retrieval, engine facts and generation with model fallback 12

List of tables

Table 1: Position of this document in the ZedPath document set 5
Table 2: Binding constraints (from ZP-DOC-03) 6

Table 3: Non-functional requirements that shape the architecture 6
Table 4: External systems and what crosses each boundary 8
Table 5: Containers and bindings (from wrangler.jsonc) 9
Table 6: Worker components (worker/) 11
Table 7: Security controls 14
Table 8: Daily budget on the Cloudflare free plan (limits verified 5 October 2026) 15
Table 9: ADR-01 Vite + React SPA on static assets with a Hono API, instead of Next.js SSR 16
Table 10: ADR-02 Read model compiled into the Worker for rules, cut-offs, routes and Journey dates 16
Table 11: ADR-03 Durable Objects for group chat 16
Table 12: ADR-04 Password key derived on the device 16
Table 13: ADR-05 Expand-then-contract database migrations 17
Table 14: ADR-06 Preset avatars generated at build time 17
Table 15: Release runbook 18
Table 16: Architectural risks 19
Table 17: Requirements traced to architectural elements 20
Table 18: Terms used in this document 21
Table 19: Sources referred to in this document 22

1 Introduction

1.1 Purpose

This document describes how ZedPath is built and deployed: the parts of the system, how they talk to each other, why each major choice was made, and how the system stays inside the Cloudflare free plan. It is written from the code as deployed, not from intentions; where something is designed but not yet running, it says so.

1.2 Scope

The whole of Releases R1 and R2: the web application (also installable as a progressive web app and wrapped by the Android app), the single Cloudflare Worker that serves it and its API, the data and AI services that Worker binds to, the offline data pipeline that compiles the read model, and the release and safety tooling around deployment.

1.3 Intended audience

Developers, reviewers and buildathon judges assessing how the system works and whether it is sound.

1.4 Relationship to other documents

Table 1: Position of this document in the ZedPath document set

Document	Relationship
ZP-DOC-01 Vision and Scope	Grandparent. Business objectives BO-1 and BO-6 (free to run, at scale) shape the platform choice
ZP-DOC-02 User Stories	Grandparent. Stories US-101 to US-204 define the walking skeleton built first
ZP-DOC-03 SRS	Parent. Constraints CON-1 to CON-5 and the non-functional requirements NFR-001 to NFR-075 are the architectural drivers
ZP-DOC-04 Use Case Model	Parent. Use-case flows become the runtime flows of Section 6
ZP-DOC-05 Data Design	Sibling. Owns the D1 schema; this document decides how the read and write paths use it
ZP-DOC-07 UI/UX, ZP-DOC-08 Test Plan	Children. Screens built on the components here; tests that prove the NFRs

2 Architectural drivers

2.1 Constraints

Table 2: Binding constraints (from ZP-DOC-03)

ID	Constraint	Architectural consequence
CON-1	Cloudflare free plan only	10 ms CPU per request, 100,000 Worker requests a day, Workers AI 10,000 neurons a day; static assets free and unlimited. Every choice below is checked against these
CON-2	TypeScript everywhere	One language across client, Worker, shared rule engine and tooling; types in `shared/` are imported by both sides
CON-3	Material Design 3	React components styled to MD3 tokens; no heavy UI framework (keeps NFR-006)
CON-4	No email, SMS or WhatsApp	Reminders are local notifications on the device (PWA and Android); account recovery uses a recovery code, not email
CON-5	Official documents cited, not redistributed	The read model holds extracted facts with source and page; Ask returns citations, not files

2.2 Quality attributes

Table 3: Non-functional requirements that shape the architecture

NFR	Requirement (summary)	How the architecture meets it
NFR-003, NFR-005	API under 500 ms; under 10 ms CPU per request	Eligibility and bands computed from a read model compiled into the Worker (no database round trip); D1 only for account data (ADR-01, ADR-02)
NFR-010, NFR-011	10,000 students per cycle; quotas below 70% on a typical day	Free-plan budget, Section 8
NFR-012	Page navigation uses no Worker requests	Static assets with `run_worker_first` limited to `/api/*`; SPA fallback for every other path
NFR-006, NFR-063	Small first download; runs in any browser, installable	Vite + React SPA, code split; service worker `public/sw.js` caches the shell
NFR-033	Secrets never in source	Wrangler secrets (`GEMINI_API_KEY`, `PEPPER`); `.dev.vars` locally, gitignored; pre-push leak scan
NFR-034, NFR-037	Ask and sign-in limits	Workers Rate Limiting bindings plus a daily allowance in D1 keyed by salted hashes
NFR-035, NFR-038	Validation and rules enforced on the server	Every write re-checked in the Worker; bound parameters only; 4xx with a field name on rejection
NFR-036	Password never leaves the device	PBKDF2-SHA256, 600,000 iterations, on the phone; server stores SHA-256 of the derived key (ADR-04)
NFR-039	Moderation fails closed	GroupRoom calls llama-guard before posting; unreadable verdict means not posted
NFR-040	Core browsing works without AI	AI is reached only by `/api/ask` and group posting; results, courses, list and Journey never call it
NFR-043	Idle group connection stays open	WebSocket hibernation with a ping/pong auto-response that does not wake the room (ADR-03)

NFR	Requirement (summary)	How the architecture meets it
NFR-050, NFR-051	Every fact cited; exact cut-offs	Read model carries source and page per fact; Z-scores stored as integers in ten-thousandths
NFR-060, NFR-062	Rules as tested pure functions; new year without code changes	Rule engine in `shared/` (pure, unit tested); the yearly read model is regenerated by `tools/`

3 System context

ZedPath has one human actor in production, the **student** (with a **senior** variant who joins groups by invitation). It depends on three external systems: **Cloudflare** (hosting, data, AI), **Google AI Studio** (Gemini models, reached only through Cloudflare AI Gateway) and **GitHub** (public source repository). The University Grants Commission (UGC) is a *source* of data, not a runtime dependency: its handbook and cut-off tables are processed offline into the read model.

Table 4: External systems and what crosses each boundary

External system	Direction	What crosses	Protection
Student's browser or Android app	In	HTTPS requests, WebSocket for group chat	Rate limits, session cookie, server-side validation
Google AI Studio (Gemini)	Out	Question, retrieved passages, engine facts (no name, contact or account id)	Through AI Gateway; key held as a Worker secret; payload logging off
GitHub	Out (developer only)	Source code	Pre-push leak scan of every new commit
UGC publications	Offline input	Handbook and cut-off PDFs	Processed on the developer laptop; facts verified by two extraction methods (NFR-052)

4 Container and deployment view

All server-side code is **one Cloudflare Worker** named `zedpath`. It serves the built single-page application as static assets and runs a Hono API only for paths under `/api/`. Everything else it needs is a binding declared in `wrangler.jsonc`.

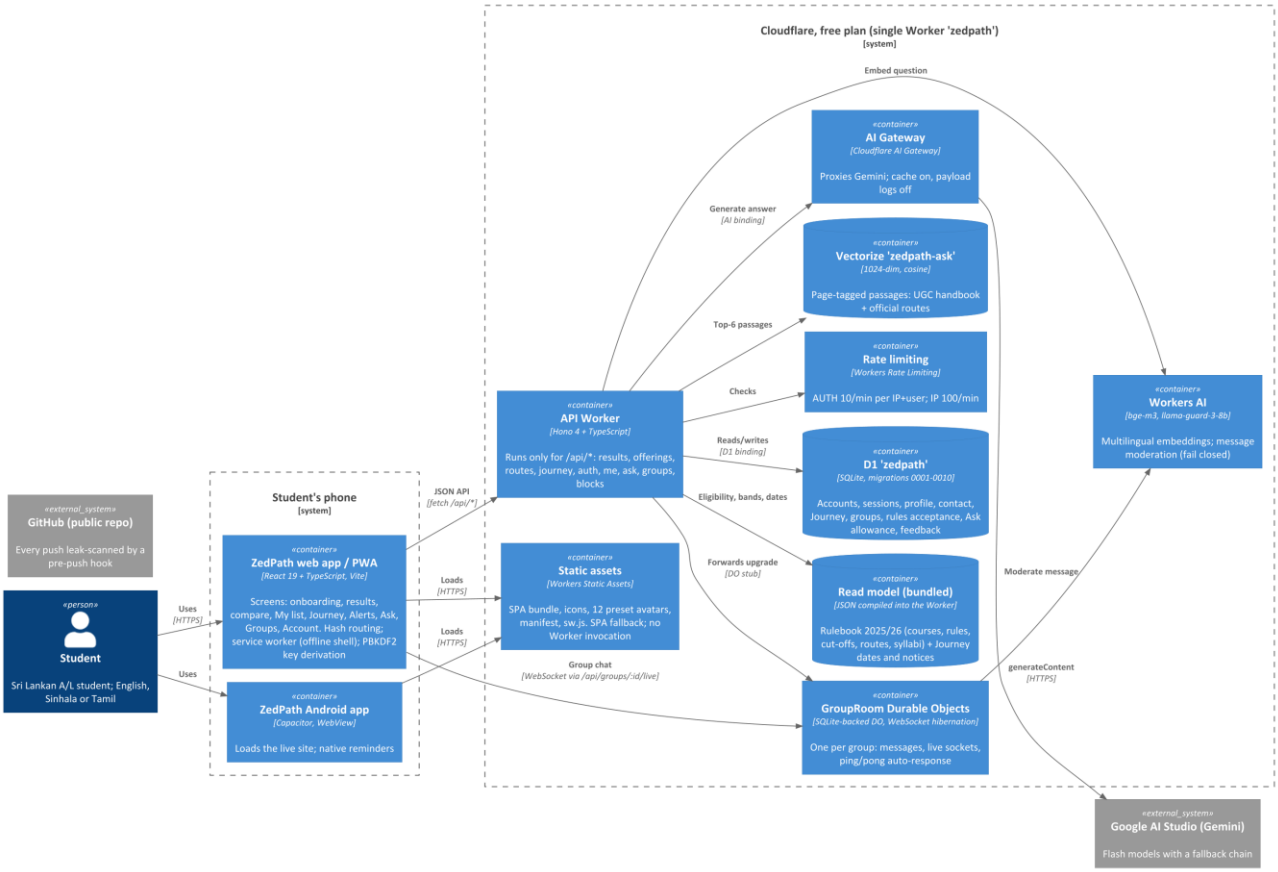


Figure 1: C4 container view of ZedPath as deployed

Table 5: Containers and bindings (from wrangler.jsonc)

Container	Technology	Binding	Role
Web app / PWA	React 19, TypeScript, Vite; hash routing; service worker	(static assets)	All screens; keeps the preference list and compare on the device; derives the password key
Android app	Capacitor WebView	(loads the live site)	Same app in its own window; native reminders
Static assets	Workers Static Assets	`assets` (SPA fallback, `run_worker_first: /api/*`)	Serves the bundle, icons, 12 preset avatars, manifest, `sw.js`; free and unlimited
API	Hono 4 on Workers	`main: worker/index.ts`	JSON API; mounts account, Journey, Ask and group routes
Read model	JSON compiled into the Worker	(bundled)	`worker/generated/rulebook-2025-2026.json` and `journey-2025-2026.json`: courses, rules, five years of cut-offs, routes, syllabi, Journey dates and notices

Container	Technology	Binding	Role
Database	Cloudflare D1 (SQLite)	`DB`	Accounts, sessions, profile, contact, Journey, groups, rules acceptance, Ask allowance, answer feedback (migrations 0001 to 0010)
Group rooms	Durable Objects, SQLite storage	`GROUP_ROOM` (class `GroupRoom`)	One object per group: messages and live WebSockets, with hibernation
Embeddings and moderation	Workers AI	`AI`	`@cf/baai/bge-m3` (multilingual, 1024 dimensions); `@cf/meta/llama-guard-3-8b` for messages
Retrieval index	Vectorize `zedpath-ask`	`ASK_INDEX`	Page-tagged passages from the UGC handbook and official routes; cosine
Model gateway	AI Gateway	via `AI.gateway()`	Proxies Gemini; cache on; payload logs off
Rate limiting	Workers Rate Limiting	`AUTH_LIMITER`, `IP_LIMITER`	10 a minute per IP and username; 100 a minute per IP

Scheduled jobs. ZP-DOC-03 provides for a weekly Cron job to check the Gazette and official notices. In this release no Cron trigger is configured: Journey dates and notices are compiled into the read model by `tools/journey` and shipped with each deploy (ADR-02). The Cron job is planned (Risk R-05).

5 Component view

5.1 Worker modules

Table 6: Worker components (worker/)

Module	Routes	Responsibility
`index.ts`	`GET /health`, `/meta`, `POST /results`, `GET /offerings/:uniCode`, `/routes`, `/journey`	Read path: eligibility and Safe/Likely/Reach bands from the bundled read model; ETag caching keyed to the build
`account.ts`	`/auth/salt`, `/auth/signup`, `/auth/login`, `/auth/recover`, `/auth/logout(-all)`, `GET/PUT/DELETE /me`, `/me/contact`, `/me/avatar`	Accounts and sessions; the `/me/*` session middleware; full account deletion including group rooms
`my-journey.ts`	`/me/journey`, `/me/journey/courses/:uni`, `/notices/:route`, `/notices/:route/applied`, `/steps/:stepId`	The student's Journey on the account; step and notice windows enforced (409)
`ask.ts`	`POST /ask`, `/ask/feedback`; localhost-only `/dev/*`	Retrieval, engine facts, Gemini generation with fallback, reply guard, daily allowance
`groups.ts`	`/groups`, `/groups/join`, `/groups/:id`, `/leave`, `/live`, `/members`, `/report`, `/blocks`	Membership and rules acceptance in D1; forwards the WebSocket upgrade to the room
`group-room.ts`	(Durable Object)	Message storage, moderation, broadcast, hibernation, ping/pong
`rulebook.ts`, `syllabi.ts`	(internal)	Load and index the read model; syllabus data and the blocked-hosts list

5.2 Client screens

The client (`src/`) is a Vite + React 19 SPA with hash routing, so every navigation stays on the static asset and never calls the Worker (NFR-012). Screens in `src/screens/`: Onboarding (results entry), Compare, MyList, Journey, Alerts, Ask, Groups, Syllabus and Account. `src/i18n/` holds English, Sinhala and Tamil strings (NFR-023). `src/pwa.ts` registers the service worker; `src/notify.ts` schedules local reminders.

5.3 Shared rule engine

`shared/` is imported by both the client and the Worker, so a rule exists exactly once. It holds the eligibility rules (`rules.ts`), banding to BR-030 to BR-036 (`banding.ts`, integer Z-scores), preference-list checks (`list.ts`), Journey windows (`journey.ts`), the key-derivation constants (`kdf.ts`, `account.ts`), group rules (`groups.ts`) and the API types (`api.ts`). Each has unit tests beside it (NFR-060, NFR-061).

6 Key runtime flows

6.1 Results to bands

1. The student enters stream, subjects, grades, district and Z-score in Onboarding.
2. The client posts the profile to `POST /api/results`.
3. The Worker validates it (`parseProfile`), then for every offering in the bundled rulebook runs the eligibility rules and the banding function. No D1 read is involved, so the request stays well under 10 ms CPU.
4. The response lists each offering with its band, years used, latest cut-off and trend, plus hidden offerings with the quoted reason and handbook page (NFR-050).

6.2 Ask with citations

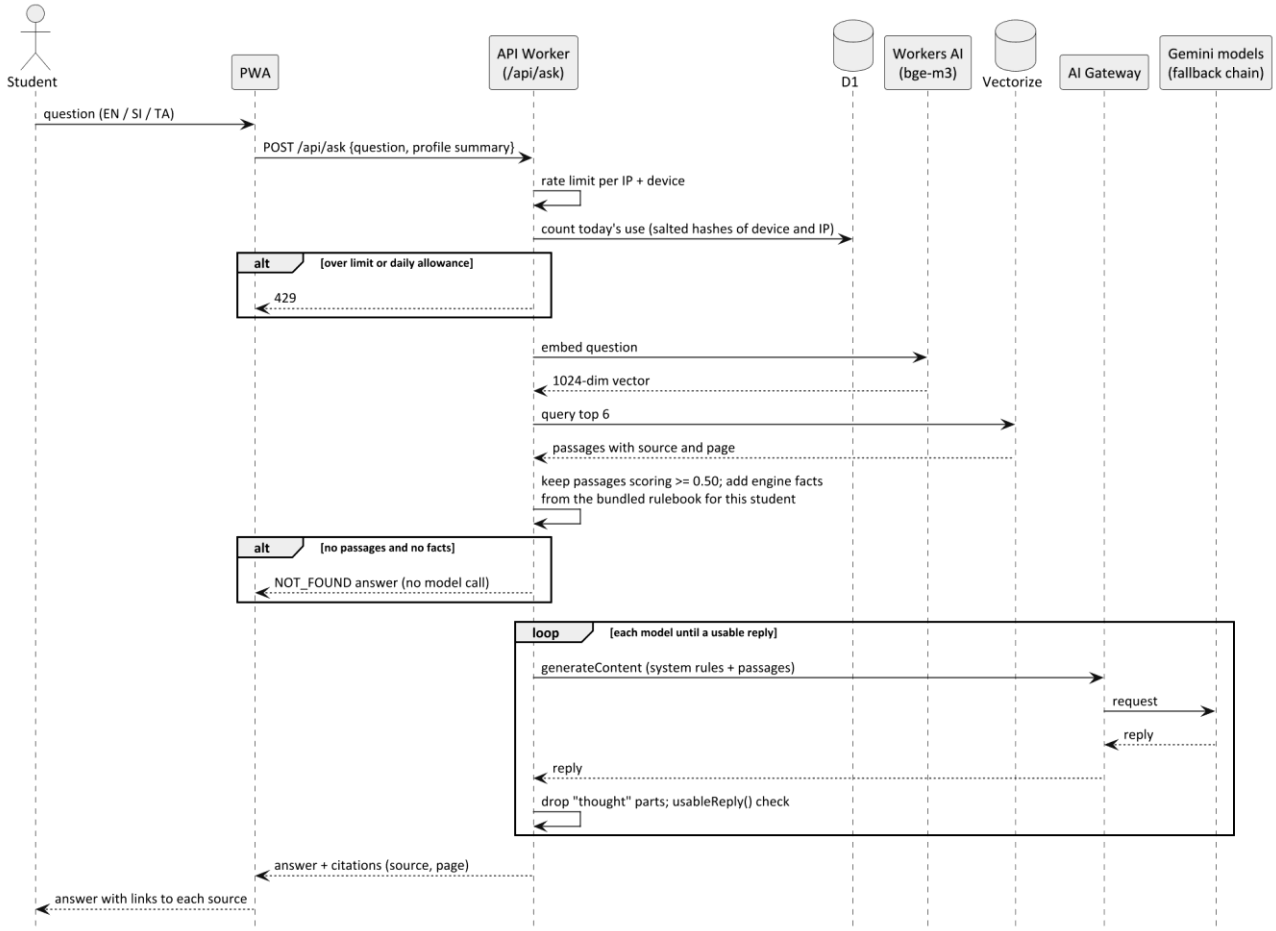


Figure 2: Ask ZedPath: retrieval, engine facts and generation with model fallback

Two guards matter. Passages below a similarity of 0.50 (calibrated on Sinhala and Tamil questions) are dropped, and when nothing relevant remains the Worker answers "not found" without calling a model. Each model reply passes `usableReply()`, which rejects a reply that talks about its own instructions; the next model in the chain (`gemini-flash-latest`, `gemini-2.5-flash`, `gemini-flash-lite-latest`, `gemini-2.5-flash-lite`) is then asked. A model that is missing, busy or out of quota (404, 429, 5xx) also falls through to the next.

6.3 Joining a group, rules accepted once

1. The client posts `POST /api/groups/join` with the group kind and, the first time, `acceptRules` set to the current rules version.
2. The Worker refuses (400) unless the rules are accepted in this request or were accepted before (`group_rules_acceptance`, one row per account and version).
3. A batch group requires results saved to the account (403 otherwise).
4. Group creation, rules acceptance and membership are written in one D1 batch with `INSERT OR IGNORE`, so a repeat join is harmless.
5. `GET /api/groups/:id/live` checks membership, then forwards the WebSocket upgrade to the group's GroupRoom, with the username added by the Worker (the client cannot claim another name).

6.4 Journey step window (409)

A Journey step can be ticked *done* only once its official window has opened. The client greys the tick out, but the rule is enforced in `PUT /api/me/journey/steps/:stepId`: the Worker computes `stepWindow()` from the bundled Journey dates and Sri Lanka's date, and returns **409** "This step has not opened yet" if it is closed. *I applied* on a notice follows the same rule (409 before the notice opens; 404 if not followed). The same shared function drives the client, so the two cannot disagree (NFR-038).

7 Security

Table 7: Security controls

Area	Control
Passwords	PBKDF2-SHA256, 600,000 iterations, run on the device; only the derived key is sent; the server stores its SHA-256 with a per-user salt derived from a server-side pepper (NFR-036, ADR-04)
Sessions	Random token in a cookie: `HttpOnly`, `Secure`, `SameSite=Lax`, path `/api`; only the token's hash is stored; *log out everywhere* deletes all sessions
Abuse	Rate limiting on sign-up, log-in and Ask; daily Ask allowance keyed by salted, day-scoped hashes (no raw IP stored)
Server-side enforcement	Validation, membership, rules acceptance, Journey windows, moderation: all in the Worker or Durable Object; the UI only mirrors them
Privacy	Contact details in their own table, never shown to others; groups show username and preset avatar only; account deletion removes every row and the member's room data (NFR-070, NFR-071, NFR-075)
Secrets	`GEMINI_API_KEY` and `PEPPER` as Wrangler secrets; `.dev.vars` locally, gitignored
Repository	Public repo; `.githooks/pre-push` runs a leak scan of every new commit (API keys, private keys, env files, forbidden account identifiers)
Deployment	A PreToolUse guard hook blocks deploys, secrets, remote migrations and resource changes unless the Worker config pins the project's own Cloudflare account and the product owner's message starts with **GO**
External content	Research reads only official domains; page text is data, never instructions; a blocked-hosts list is kept in the code

8 Free-plan budget

Table 8: Daily budget on the Cloudflare free plan (limits verified 5 October 2026)

Resource	Free limit	Use per student session (estimate)	Typical day (1,000 sessions)	Results day (5,000 sessions)
Worker requests	100,000 a day	about 8 API calls (pages are static and free)	8,000 (8%)	40,000 (40%)
Worker CPU	10 ms a request	read path about 1 to 3 ms; PBKDF2 runs on the phone, not here	within limit	within limit
Static assets	unlimited	all page loads	free	free
D1	5 million rows read, 100,000 rows written a day	about 20 read, 5 written	well under 10%	under 25%
Workers AI	10,000 neurons a day	bge-m3 embedding per question; llama-guard per group message	low; Ask capped at 30 a device a day	watch item (Risk R-02)
Durable Objects	free with SQLite storage	hibernating rooms cost nothing while idle	low	low
Vectorize	free tier	one query per question	low	low

The figures are planning estimates; ZP-DOC-08 defines the measurement that checks NFR-011.

9 Architecture decision records

Table 9: ADR-01 Vite + React SPA on static assets with a Hono API, instead of Next.js SSR

Field	Record
Status	Accepted, approved by the product owner on 5 October 2026
Context	The deck proposed Next.js. Server rendering on Workers runs on every page view and risks exceeding the free plan's 10 ms CPU (Error 1102) and its request quota
Decision	Build a static SPA served as Workers Static Assets, plus a small Hono API in the same Worker that runs only for `/api/*`
Consequences	Page views are free and fast; one deployable unit. No server rendering, so the first screen relies on a small bundle (NFR-006)

Table 10: ADR-02 Read model compiled into the Worker for rules, cut-offs, routes and Journey dates

Field	Record
Status	Accepted
Context	Banding every offering for a student would need many D1 reads per request; the data changes once a year (dates a few times)
Decision	`tools/` compile verified data into JSON bundled with the Worker; D1 holds only per-student data. ETags keyed to the build let clients revalidate cheaply
Consequences	Read path needs no database and fits the CPU limit. A data change needs a deploy; the D1 system-of-record schema (ZP-DOC-05) remains the design for later live curation

Table 11: ADR-03 Durable Objects for group chat

Field	Record
Status	Accepted
Context	Live group chat needs one ordered place per group and long-lived connections, on the free plan
Decision	One SQLite-backed `GroupRoom` Durable Object per group, using the WebSocket hibernation API and a `ping`/`pong` auto-response so keep-alives never wake the room. Posts are processed one at a time and moderated by llama-guard, failing closed
Consequences	Idle rooms cost nothing; ordering is natural. Membership stays in D1 and is checked by the Worker before the upgrade is forwarded

Table 12: ADR-04 Password key derived on the device

Field	Record
Status	Accepted
Context	A strong password hash (PBKDF2 at 600,000 iterations) would exceed 10 ms CPU on the Worker
Decision	The phone derives the key with PBKDF2-SHA256 using a salt from `/auth/salt`; the server hashes the received key once with SHA-256
Consequences	The password never leaves the device and the Worker stays within its CPU limit. Older phones take about a second to sign in

Table 13: ADR-05 Expand-then-contract database migrations

Field	Record
Status	Accepted
Context	Remote migrations run before the deploy that needs them, so old and new Worker versions briefly share one schema
Decision	A migration only adds; removal happens in a later migration once no deployed version reads the old shape. Example: 0010 moved rules acceptance to `group_rules_acceptance` but keeps `group_member.rules_version`
Consequences	No downtime and a safe rollback to the previous version; a withdrawn column lingers until a contract step

Table 14: ADR-06 Preset avatars generated at build time

Field	Record
Status	Accepted
Context	Students need a picture in groups, but uploaded photos would carry privacy and moderation risk (NFR-071)
Decision	`tools/avatars/make-avatars.ts` generates 12 SVG avatars into `public/avatars/`; the account stores only a choice. No outside image service is called
Consequences	No uploads, no third-party calls, served free as static assets

10 Deployment and rollback

Table 15: Release runbook

Step	Action
1	`npm run build` (TypeScript build and Vite bundle) is green and `npm test` passes
2	The bundle is checked locally with `wrangler dev`, key flow clicked through
3	`wrangler whoami` confirms the ZedPath Cloudflare account (the pinned account in `wrangler.jsonc`), shown to the product owner
4	Remote D1 migrations, if any, are applied first (expand-only, ADR-05)
5	The product owner replies with a message starting **GO**; the guard hook blocks the deploy otherwise
6	Deploy; record the version id Wrangler prints
7	Verify on the live URL through a channel that can see it
8	Log version, URL and change in `WORKLOG.md`; commit and push (leak-scanned)

Rollback. The previous version id is written down before deploying. `wrangler rollback <previous-version-id>` restores it (also GO-gated). Because migrations only expand, the earlier version still works on the newer schema.

11 Risks

Table 16: Architectural risks

ID	Risk	Mitigation
R-01	Results-day spike exhausts the 100,000 daily requests	Pages are static; ETag revalidation; IP rate limit; budget monitored in Cloudflare observability
R-02	Workers AI neuron quota exhausted by Ask and moderation	Per-device and per-IP daily allowance; "not found" answers skip the model; core browsing does not depend on AI (NFR-040)
R-03	Gemini model retired or rate limited	Four-model fallback chain through AI Gateway; clear 503 message
R-04	Model returns reasoning instead of an answer	`usableReply()` guard and thought-part filtering, with tests
R-05	Gazette and notice changes missed between deploys	Planned weekly Cron job; until then notices are curated and redeployed; each shows its retrieval date
R-06	Deploy to a wrong Cloudflare account	Account pinned in config; guard hook checks the pin and requires GO
R-07	Secret leaked to the public repo	Pre-push leak scan of all new commits; rotate first if a hit ever reaches a commit

Appendix A: Traceability: requirements to architecture

Table 17: Requirements traced to architectural elements

Requirement	Element	Section
CON-1, NFR-005, NFR-010, NFR-011	ADR-01, ADR-02, free-plan budget	2, 8, 9
NFR-012, NFR-063	Static assets, SPA fallback, service worker	4, 5.2
NFR-033, NFR-036, NFR-037	Security controls, ADR-04	7, 9
NFR-035, NFR-038	Worker enforcement, Journey 409, group join	6.3, 6.4
NFR-039, NFR-043, NFR-071	GroupRoom, ADR-03, ADR-06	4, 9
NFR-040, NFR-050	Ask flow, read model with citations	6.2, ADR-02
NFR-060, NFR-061, NFR-062	Shared rule engine, `tools/` pipeline	5.3, ADR-02
FE-6 Ask	`ask.ts`, Vectorize, AI Gateway	5.1, 6.2
FE-7 People around the student	`groups.ts`, `group-room.ts`	5.1, 6.3
FE-4 Journey and deadlines	`my-journey.ts`, `shared/journey.ts`	5.1, 6.4

Appendix B: Glossary

Table 18: Terms used in this document

Term	Meaning
ADR	Architecture Decision Record: a short record of one decision, its context and consequences
AI Gateway	Cloudflare service that proxies calls to AI providers, with caching and analytics
C4 model	A way of drawing software at four levels: context, containers, components, code
D1	Cloudflare's serverless SQL database, built on SQLite
Durable Object	A Cloudflare object with a single location and its own storage, used here for one chat room per group
Hibernation	Durable Object WebSocket mode in which an idle object is evicted while its connections stay open
Hono	A small web framework for Workers
PBKDF2	Password-Based Key Derivation Function 2: a deliberately slow, salted hash
PWA	Progressive web app: a website that can be installed and work offline
Read model	A query-optimised copy of data derived from the system of record
SPA	Single-page application
Vectorize	Cloudflare's vector database, used to find passages similar to a question

Appendix C: References

Table 19: Sources referred to in this document

Ref	Source
[1]	S. Brown, *The C4 model for visualising software architecture*, c4model.com
[2]	M. Nygard, *Documenting Architecture Decisions*, 2011
[3]	Cloudflare, *Workers limits*, *Static Assets*, *D1 limits*, *Durable Objects: WebSocket hibernation*, *Workers AI pricing*, developers.cloudflare.com, retrieved 5 October 2026
[4]	ZP-DOC-03 Software Requirements Specification; ZP-DOC-05 Data Design
[5]	`wrangler.jsonc`, `worker/`, `shared/`, `src/`, `migrations/` in the ZedPath repository